PORTAL HỌ LÊ VĂN - PHÁI 4 - CHI 2

Thôn An Lợi, Xã Triệu Bình, Tỉnh Quảng Trị



Lê Gia Khánh

Contents

1 Giới thiệu & Kiến trúc **2**
1.1 Giới thiệu dự án 2
1.2 Kiến trúc hệ thống 2

2 Xây dựng Frontend **3**
2.1 Kiến trúc Frontend 4
2.2 Mã nguồn Frontend 4

3 Xây dựng Backend **6**
3.1 Kiến trúc Backend 6
3.2 Mã nguồn Backend 7

4 Xây dựng RAG Chatbot **8**
4.1 Kiến trúc RAG Chatbot 8
4.2 Mã nguồn RAG Chatbot 9

5 Đóng gói & Triển khai **10**
5.1 Đóng gói & Khởi chạy Local bằng Docker 10
5.2 Triển khai Cloud bằng Supabase & Render & Vercel 11

1 Giới thiệu & Kiến trúc

1.1 Giới thiệu dự án

Dự án **Portal Họ Lê Văn - Phái 4 - Chi 2** là nền tảng số hóa được xây dựng nhằm bảo tồn phả hệ, lưu trữ tư liệu lịch sử và kết nối các thế hệ con cháu trong dòng họ. Website không chỉ là nơi lưu giữ gia phả số, tra cứu lịch giỗ chạp truyền thống của Chi 2 - Phái 4 - Họ Lê Văn (thôn An Lợi, xã Triệu Bình, tỉnh Quảng Trị) mà còn là kênh truyền thông cập nhật tin tức, sự kiện nội bộ kịp thời.

Hệ thống sở hữu giao diện hiện đại, tối ưu hiển thị trên đa thiết bị và tích hợp các công nghệ nổi bật như Trợ lý AI (RAG Chatbot tra cứu thông tin dòng họ), Bản đồ số định vị (Từ đường & Lăng mộ) cùng Cây gia phả trực quan, mang lại trải nghiệm tra cứu thuận tiện và sống động.

Thông tin truy cập dự án:

- Website: Portal Họ Lê Văn - Phái 4 - Chi 2
- GitHub: GitHub - Portal-HoLeVan-Phai4-Chi2



Figure 1: Giao diện trang chủ Portal Họ Lê Văn - Phái 4 - Chi 2

1.2 Kiến trúc hệ thống

Hệ thống **Portal Họ Lê Văn - Phái 4 - Chi 2** được xây dựng theo mô hình kiến trúc phân tách đa tầng (*Decoupled Multi-Tier Architecture*), tối ưu hóa hiệu năng và đảm bảo khả năng mở rộng linh hoạt. Kiến trúc tổng thể được phân chia thành 4 tầng chính:

- **Tầng Giao diện (Client Tier):** Xây dựng bằng React 18, Vite, TypeScript và Tailwind CSS. Tích hợp thư viện *React Flow* (tăng tốc GPU) để hiển thị cây gia phả đa tầng mượt mà, cùng *Leaflet GIS* định vị bản đồ vệ tinh tương tác. Có cơ chế tự động nạp dữ liệu ngoại tuyến (*Offline Fallback*) khi xảy ra sự cố mạng.
- **Tầng Dịch vụ (Backend API Tier):** Vận hành trên nền tảng Node.js (Express.js & TypeScript), cung cấp các RESTful API phục vụ nghiệp vụ gia phả, tin tức và sự kiện.

Đảm nhiệm xác thực người dùng qua JWT, phân quyền theo vai trò (RBAC) và xử lý luồng dữ liệu trước khi nạp vào AI.

- **Tầng Trí tuệ nhân tạo (AI RAG Tier):** Kết hợp mô hình ngôn ngữ lớn Google Gemini cùng kỹ thuật *RAG (Retrieval-Augmented Generation)*. Hệ thống trích xuất dữ liệu phả hệ chuẩn xác làm ngữ cảnh (*Context*), giúp chatbot trả lời thông minh bằng tiếng Việt với cam kết không bịa đặt thông tin (*Zero-Hallucination*).
- **Tầng Dữ liệu & Lưu trữ (Data Tier):** Sử dụng cơ sở dữ liệu quan hệ PostgreSQL để quản lý cấu trúc phả hệ, thế thứ và tài khoản; kết hợp Cloud Object Storage để lưu trữ hình ảnh chân dung và tư liệu lịch sử dòng họ.

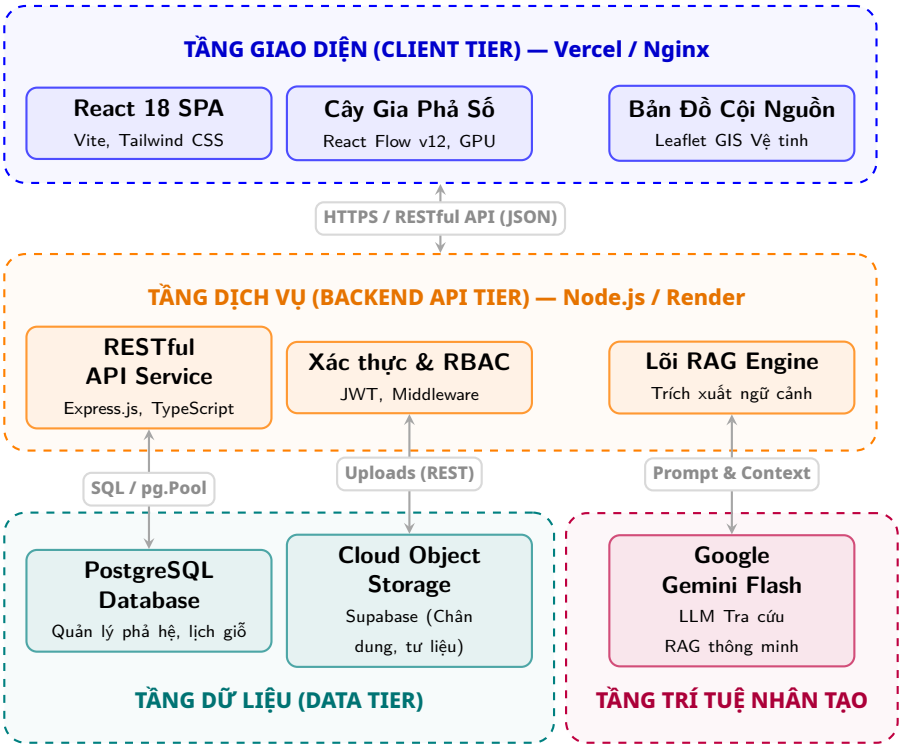


Figure 2: Sơ đồ kiến trúc phân tách đa tầng hệ thống Portal Họ Lê Văn

Dự án được tổ chức theo mô hình *Monorepo* phân tách rõ ràng giữa các phân hệ độc lập, tạo tiền đề thuận lợi cho việc phát triển, kiểm thử và triển khai:

```
Portal-HoLeVan-Phai4-Chi2/
  frontend/           # Ứng dụng giao diện Client (React 18, Vite, Tailwind CSS)
  backend/            # Máy chủ dịch vụ API chính (Node.js, Express, TypeScript)
  rag-service/        # Dịch vụ Trí tuệ nhân tạo RAG (Python, FastAPI, Vector DB)
  scripts/            # Kịch bản bảo trì, đồng bộ và nạp dữ liệu phả hệ
  docker-compose.yml  # Cấu hình điều phối và khởi chạy toàn bộ hệ thống
  README.md           # Tài liệu hướng dẫn kỹ thuật và triển khai dự án
```

Figure 3: Cấu trúc thư mục mã nguồn mức tổng thể của dự án

2 Xây dựng Frontend

2.1 Kiến trúc Frontend

Giao diện người dùng của hệ thống **Portal Họ Lê Văn - Phái 4 - Chi 2** được phát triển theo mô hình ứng dụng đơn trang (*Single Page Application - SPA*), sử dụng các công nghệ hiện đại nhằm tối ưu tốc độ phản hồi và trải nghiệm người dùng:

- **Khung phát triển chính:** Sử dụng **React 18** kết hợp với **TypeScript** và bộ công cụ đóng gói siêu tốc **Vite**, giúp kiểm soát chặt chẽ kiểu dữ liệu, giảm thiểu lỗi runtime và tối ưu hóa thời gian tải trang.
- **Thiết kế giao diện:** Áp dụng **Tailwind CSS** với hệ màu mang đậm nét văn hóa truyền thống (Đỏ đô, Vàng hoàng gia, Kem sáng). Giao diện đáp ứng (*Responsive*) 100% trên máy tính, máy tính bảng và điện thoại thông minh.
- **Cây Gia Phả Tương Tác (Phân hệ cốt lõi):** Tích hợp thư viện chuyên dụng **React Flow v12**, áp dụng thuật toán tính toán tọa độ tự động theo tầng thế hệ, hiển thị đa hôn phối (Chánh phối, Thứ phối) và tăng tốc phần cứng qua GPU (`transform: translateZ(0)`) đạt tốc độ khung hình mượt mà 60–120 FPS trên di động.
- **Bản Đồ Số Vệ Tinh (GIS):** Ứng dụng thư viện **Leaflet** tích hợp lớp bản đồ vệ tinh độ phân giải cao, định vị chính xác Nhà Thờ Họ và Khu Lăng Mộ Cồn Giữa kèm chức năng điều hướng GPS qua Google Maps.
- **Quản lý trạng thái & Mạng:** Quản lý phiên đăng nhập và phân quyền bằng **Zustand** (`authStore`); giao tiếp API thông qua **Axios** với cơ chế nạp dữ liệu ngoại tuyến dự phòng (*Offline Fallback*) khi máy chủ gặp sự cố.

2.2 Mã nguồn Frontend

Mã nguồn thư mục `frontend/src` được tổ chức theo cấu trúc phân tầng chức năng rõ ràng, tách biệt giữa giao diện (*components*), nghiệp vụ trang (*pages*), dịch vụ mạng (*services*) và dữ liệu:

```
frontend/src/
   components/           # Các khối giao diện tái sử dụng
       Auth/             # Modal đăng nhập, đăng ký và làm bài test nâng cấp vai trò
       Chatbot/          # Khung chat nổi hỏi đáp thông minh với AI RAG
       FamilyTree/       # Bộ thành phần dựng cây gia phả, node và đường liên kết
       Home/             # Các section trang chủ (Bản đồ từ đường, Lời tác giả)
       Layout/           # Thanh điều hướng (Navbar) và chân trang (Footer)
       News/             # Thẻ hiển thị tin tức và trình soạn thảo văn bản
   pages/                # Các trang màn hình chính của ứng dụng
   services/             # Lớp kết nối HTTP gọi RESTful API từ Backend
   store/                # Quản lý trạng thái toàn cục ứng dụng (Zustand)
   utils/                # Tiện ích bổ trợ (tính toán Âm lịch, cắt ảnh chân dung)
   types/                # Khai báo cấu trúc kiểu dữ liệu TypeScript dùng chung
   data/                 # Dữ liệu JSON tĩnh dự phòng cho chế độ Offline Fallback
   App.tsx               # Định tuyến (Routing) và cấu trúc khung nhìn ứng dụng
   main.tsx              # Điểm khởi tạo nạp React vào DOM gốc
   index.css             # Định nghĩa CSS toàn cục, quy tắc font và tối ưu GPU
```

Figure 4: Cấu trúc thư mục mã nguồn phía Client (frontend/src)

Mô tả chức năng chi tiết các tệp tin quan trọng:

- **Phân hệ Màn hình chính (`pages/`):**
  - `HomePage.tsx`: Trang chủ cổng thông tin, hiển thị đại cảnh Từ đường, câu đối truyền thống, liên kết nhanh và bản đồ vệ tinh.
  - `FamilyTreePage.tsx`: Không gian số hóa gia phả, cho phép tương tác zoom/pan, tìm kiếm thành viên và quản lý phả hệ trực quan.
  - `MemorialCalendarPage.tsx`: Sổ kỵ nhật số 12 tháng Âm lịch, tự động tính toán ngày giỗ tiền nhân theo phong tục họ tộc.
  - `NewsPage.tsx` & `NewsDetailPage.tsx`: Quản lý và hiển thị danh sách bài viết, thông báo, tư liệu lịch sử dòng họ.
  - `MapPage.tsx`: Trang chuyên biệt hiển thị bản đồ số và chỉ đường tận nơi về Từ đường và Lăng mộ.
- **Phân hệ Cây Gia phả (`components/FamilyTree/`):**
  - `TreeCanvas.tsx`: Trái tim của cây phả hệ, xử lý thuật toán phân tầng theo đời, căn chỉnh vị trí node tránh chồng lấn và tối ưu cuộn mượt.
  - `MemberNode.tsx`: Khối hiển thị thông tin chi tiết từng thành viên (ảnh đại diện, đời thứ, năm sinh/mất, danh xưng phối ngẫu).
  - `CustomFamilyEdge.tsx` & `CustomSpouseStraightEdge.tsx`: Các đường vẽ kết nối tùy biến biểu diễn quan hệ huyết thống trực hệ và hôn phối.
  - `MemberFormModal.tsx`: Biểu mẫu nhập liệu thêm/sửa thông tin thành viên kèm công cụ cắt ảnh tỷ lệ 1:1.

- **Phân hệ Tương tác & Hỗ trợ (`components/`, `services/`, `utils/`):**
  - `ChatbotPanel.tsx`: Giao diện trợ lý ảo AI nổi ở góc phải, gửi nhận câu hỏi và hiển thị câu trả lời từ mô hình RAG.
  - `AncestralMapSection.tsx`: Tích hợp bản đồ Leaflet vệ tinh, chuyển đổi vị trí giữa Nhà thờ và Lăng mộ Cồn Giữa.
  - `RichDocEditor.tsx`: Trình soạn thảo văn bản giàu định dạng (HTML WYSIWYG) hỗ trợ Quản trị viên đăng tin tức kèm ảnh.
  - `api.ts`: Cấu hình Axios Client, tự động đính kèm JWT Bearer Token vào Header cho các yêu cầu bảo mật.
  - `authStore.ts`: Quản lý trạng thái người dùng (Khách, Thành viên tiêu chuẩn, Ưu tú, Quản trị viên) bằng Zustand.
  - `lunarCalendar.ts`: Thuật toán thiên văn chuyển đổi ngày Dương lịch sang Âm lịch chính xác từ năm 1900 đến 2100.

3 Xây dựng Backend

3.1 Kiến trúc Backend

Hệ thống máy chủ dịch vụ của **Portal Họ Lê Văn - Phái 4 - Chi 2** được phát triển theo mô hình kiến trúc phân lớp (*Layered Architecture*) kết hợp nguyên lý thiết kế RESTful API, vận hành trên nền tảng đám mây đảm bảo tính sẵn sàng cao:

- **Nền tảng & Môi trường thực thi:** Xây dựng trên **Node.js 22 LTS** kết hợp framework **Express.js** và **TypeScript**. Kiến trúc hướng module giúp tách biệt rõ ràng giữa các tầng: *Routes* (Định tuyến API) $\rightarrow$ *Controllers* (Điều phối nghiệp vụ) $\rightarrow$ *Services & Database* (Xử lý dữ liệu).
- **Cơ sở dữ liệu & Connection Pooling:** Sử dụng hệ quản trị cơ sở dữ liệu quan hệ **PostgreSQL**. Ứng dụng cơ chế quản lý kết nối `pg.Pool` kết hợp mã hóa SSL, hỗ trợ tự động đồng bộ lược đồ bảng (*Schema*) và nạp sẵn dữ liệu ban đầu (*Auto-seeding*) cho 321 thành viên và các bài viết tư liệu.
- **Bảo mật & Phân quyền người dùng (RBAC):** Xác thực phiên làm việc không trạng thái (*Stateless*) bằng **JSON Web Token (JWT)**, mã hóa mật khẩu an toàn bằng **bcryptjs**. Hệ thống phân quyền chặt chẽ theo 3 cấp bậc: Thành viên tiêu chuẩn (*Standard*), Thành viên ưu tú (*Elite*) và Quản trị viên tối cao (*Admin*). Toàn bộ API được bảo vệ bởi **Helmet** và cấu hình **CORS** an toàn.
- **Xử lý tệp tin & Lưu trữ đám mây:** Tích hợp middleware **Multer** kết hợp tải lên trực tiếp **Supabase Storage Bucket**, tự động tối ưu hóa và phục vụ hình ảnh qua mạng lưới phân phối nội dung (CDN) toàn cầu.
- **Lõi RAG Engine tích hợp (In-Backend RAG):** Dịch vụ AI Chatbot được tích hợp trực tiếp trong Backend (`ragService.ts`), liên tục đọc kho tri thức dòng họ định dạng

Markdown và kết nối với Google Gemini API để tạo câu trả lời chính xác, độ trễ thấp.

3.2 Mã nguồn Backend

Toàn bộ mã nguồn máy chủ nằm trong thư mục `backend/src/`, được tổ chức khoa học theo từng nhóm chức năng độc lập:

```
backend/src/
  config/              # Cấu hình CSDL, lược đồ SQL và kịch bản nạp dữ liệu mẫu
  controllers/         # Tầng tiếp nhận request và điều phối logic nghiệp vụ
  middleware/          # Tầng kiểm soát xác thực JWT, phân quyền và upload file
  routes/              # Khai báo các điểm cuối RESTful API theo tài nguyên
  services/            # Dịch vụ nghiệp vụ độc lập (Lõi RAG Engine & AI Chat)
  types/               # Khai báo cấu trúc kiểu dữ liệu TypeScript (User, Member, News)
  data/                # Dữ liệu nguồn và kho tri thức số hóa dòng họ
     members.json      # Danh sách gốc 321 thành viên gia phả
     news.json         # Danh sách bài viết, tư liệu mẫu
     knowledge/        # Tài liệu Markdown chuyên sâu phục vụ RAG Chatbot
  server.ts            # Điểm khởi chạy chính của Express HTTP Server
```

Figure 5: Cấu trúc thư mục mã nguồn máy chủ API (backend/src)

Mô tả chức năng chi tiết các tệp tin quan trọng:

- **Phân hệ Khởi tạo & Cơ sở dữ liệu (`config/`, `server.ts`):**
  - `server.ts`: Điểm khởi động ứng dụng Express, thiết lập các middleware toàn cục (CORS, Helmet, Body Parser), đăng ký router và kích hoạt kiểm tra sức khỏe hệ thống (`/health`).
  - `db.ts`: Thiết lập kết nối PostgreSQL qua `pg.Pool` (hỗ trợ SSL), tự động kiểm tra kết nối và khởi tạo các bảng dữ liệu nếu chưa tồn tại.
  - `initSql.ts` & `init_postgres.sql`: Định nghĩa lược đồ cấu trúc (*DDL Schema*) của 3 bảng cốt lõi: `users`, `members` và `news`.
  - `seed_members.ts` & `seed_news.ts`: Tự động đọc dữ liệu JSON để nạp 321 thành viên và các bài viết mẫu vào CSDL khi khởi động hệ thống lần đầu.
- **Phân hệ Điều phối nghiệp vụ (`controllers/`):**
  - `authController.ts`: Xử lý đăng ký, đăng nhập, cấp phát JWT, cập nhật hồ sơ và chấm điểm bài test nâng cấp vai trò thành viên.
  - `membersController.ts`: Quản lý dữ liệu phả hệ (CRUD thành viên, liên kết cha/mẹ, vợ/chồng, lọc thế hệ và tra cứu nhanh).
  - `memorialsController.ts`: Truy vấn và tính toán động danh sách các ngày giỗ theo từng tháng Âm lịch dựa trên thông tin thành viên đã khuất.
  - `newsController.ts`: Quản lý tin tức, tư liệu, sự kiện (đăng tải bài viết, cập nhật ảnh bìa, đếm lượt xem).

- `chatController.ts`: Tiếp nhận câu hỏi của người dùng từ khung chat, chuyển tiếp sang RAG Service để phân tích và trả lời.

- **Phân hệ Trung gian & Định tuyến (`middleware/`, `routes/`):**
  - `auth.ts`: Chứa các middleware bảo mật: `verifyToken` (giải mã JWT), `requireAdmin` (kiểm tra quyền quản trị), `requireEliteOrAdmin` (kiểm tra quyền viết bài).
  - `upload.ts`: Xử lý tải ảnh đại diện và ảnh bài viết, tự động đẩy lên Supabase Storage CDN và trả về đường dẫn URL vĩnh viễn.
  - `auth.ts`, `members.ts`, `news.ts`, `memorials.ts`, `chat.ts`: Các tệp định tuyến map các phương thức HTTP (GET, POST, PUT, DELETE) với các hàm xử lý tương ứng trong controller.

- **Phân hệ Trí tuệ nhân tạo RAG (`services/ragService.ts`):**
  - `ragService.ts`: Lõi xử lý RAG độc lập, tự động đọc 3 tệp tri thức (`gia_pha_chi_tiet_ho_le_van.md`, `lich_gio_ky_va_an_tang.md`, `tong_quan_va_thong_ke_dong_ho.md`), xây dựng Prompt chuẩn kèm chỉ dẫn nghiêm ngặt (*System Prompt*) và gửi yêu cầu tới mô hình Google Gemini để nhận kết quả phản hồi chính xác tuyệt đối.

4 Xây dựng RAG Chatbot

4.1 Kiến trúc RAG Chatbot

Để hỗ trợ con cháu dòng họ tra cứu phả hệ nhanh chóng bằng ngôn ngữ tự nhiên mà vẫn đảm bảo tính chính xác lịch sử tuyệt đối, hệ thống tích hợp công nghệ **RAG (Retrieval-Augmented Generation - Tạo sinh tăng cường bằng truy xuất dữ liệu)**.

Nguyên lý hoạt động và cam kết **Zero-Hallucination**: Mô hình ngôn ngữ lớn (LLM) thông thường có xu hướng "ảo giác" (*Hallucination*) khi hỏi về các dữ liệu tư nhân hoặc gia phả nội bộ. RAG giải quyết triệt để vấn đề này bằng cách:

1. Không để mô hình tự suy đoán dữ liệu;
2. Khi nhận câu hỏi, hệ thống tự động truy xuất các đoạn thông tin phả hệ chính xác nhất làm ngữ cảnh (*Context*);
3. Ràng buộc mô hình bằng *System Prompt* nghiêm ngặt: chỉ được trả lời dựa trên ngữ cảnh được cung cấp, xưng hô tôn kính (*"Cụ", "Ông", "Bà"*), và từ chối nếu dữ liệu chưa ghi nhận.

Sơ đồ luồng xử lý RAG (RAG Pipeline Workflow): Quy trình từ lúc người dùng đặt câu hỏi đến khi nhận câu trả lời diễn ra qua 4 giai đoạn khép kín được minh họa trong Hình 6:

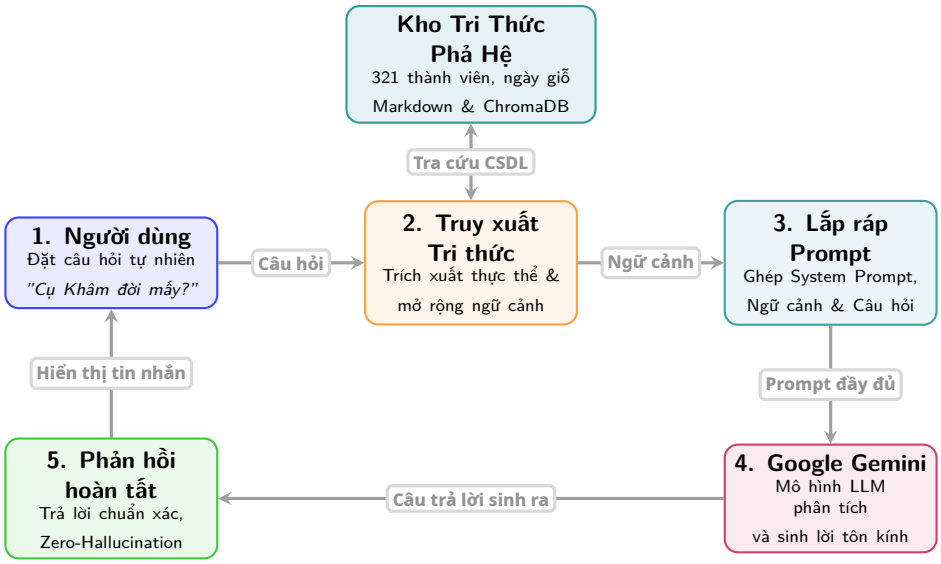


Figure 6: Sơ đồ luồng xử lý truy xuất và tạo sinh câu trả lời của RAG Chatbot

4.2 Mã nguồn RAG Chatbot

Phân hệ RAG được triển khai dưới dạng dịch vụ chuyên sâu tại thư mục `rag-service/` (Python FastAPI & ChromaDB) kết hợp cùng module `ragService.ts` tích hợp sẵn trong Node.js Backend phục vụ môi trường Cloud 24/7:

```
rag-service/
   data/
      raw_documents/          # Kho tài liệu tri thức số hóa định dạng Markdown
         gia_pha_chi_tiet_ho_le_van.md      # Chi tiết 321 thành viên từ Đời 9-16
         lich_gio_ky_va_an_tang.md          # Sổ ky nhật Âm lịch và vị trí lăng mộ
         tong_quan_va_thong_ke_dong_ho.md   # Khởi nguyên An Lợi, quy ước thế thứ
      vector_store/           # Cơ sở dữ liệu Vector ChromaDB lưu trữ trên đĩa
   scripts/
      generate_rag_documents.py # Script tự động sinh tài liệu Markdown từ CSDL
   src/
      config.py               # Quản lý biến môi trường và thiết lập tham số
      ingest.py               # Chunking văn bản và nạp vector vào ChromaDB
      main.py                 # Điểm khởi chạy FastAPI server và định nghĩa API
      models.py               # Schema Pydantic kiểm tra tính hợp lệ dữ liệu
      rag_pipeline.py         # Lõi LangChain: Prompt Template, Retriever & LLM
   requirements.txt           # Danh mục các thư viện AI (LangChain, FastAPI, Chroma)
   Dockerfile                 # Đóng gói microservice chạy độc lập trên Docker
```

Figure 7: Cấu trúc thư mục phân hệ Trí tuệ nhân tạo (rag-service/)

Mô tả chức năng chi tiết các tệp tin quan trọng:

- **Phân hệ Lõi Dịch vụ AI (`rag-service/src/`):**
  - `main.py`: Khởi chạy ứng dụng FastAPI, cấu hình CORS, thiết lập các endpoint xử lý hội thoại đồng bộ (`/chat/sync`) và truyền phát thời gian thực qua Server-Sent Events (`/chat`).

- `rag_pipeline.py`: Trái tim của microservice, nạp mô hình ngôn ngữ lớn (Google Gemini 1.5 Flash), cấu hình bộ truy xuất tương đồng (*Chroma Retriever top-k=6*) và lắp ghép ngữ cảnh vào khuôn mẫu câu hỏi.
- `ingest.py`: Tự động đọc các tệp tri thức Markdown, sử dụng `RecursiveCharacterTextSplitter` chia nhỏ văn bản thành các đoạn 500 ký tự (độ gối đầu 50 ký tự), chuyển đổi sang vector nhúng qua mô hình `text-embedding-004` và lưu bền vững vào ChromaDB.
- `models.py`: Định nghĩa các lớp dữ liệu Pydantic (`ChatRequest`, `ChatResponse`) đảm bảo chuẩn hóa dữ liệu giao tiếp với Frontend.
- `config.py`: Quản lý tập trung các khóa bí mật (`GEMINI_API_KEY`), tham số nhiệt độ (*temperature*), cổng kết nối và đường dẫn dữ liệu.

- **Phân hệ Chuẩn bị Tri thức & Kịch bản (`scripts/`, `data/`):**
  - `generate_rag_documents.py`: Tiện ích phân tích cơ sở dữ liệu phả hệ hơn 321 thành viên, tự động thiết lập liên kết thân tộc cha/mẹ/vợ/chồng và sinh ra 3 tệp Markdown tri thức chuẩn xác.
  - `gia_pha_chi_tiet_ho_le_van.md`: Chứa thông tin chi tiết từng thành viên trải dài qua 8 thế hệ tại Chi 2 (từ Đời 9 đến Đời 16 của Phái 4 Họ Lê Văn).
  - `lich_gio_ky_va_an_tang.md`: Bảng tra cứu ngày mất, ngày kỵ nhật Âm lịch của hơn 109 vị tiền nhân cùng địa điểm an táng (Cồn Giữa, Cồn Cát, Ba Lăng,...).
  - `tong_quan_va_thong_ke_dong_ho.md`: Lược sử khởi lập làng An Lợi, quy ước tính thế thứ và các số liệu thống kê nhân khẩu của dòng họ.
- **Phân hệ RAG Phục vụ Production (`backend/src/services/`):**
  - `ragService.ts`: Phiên bản RAG Engine viết bằng TypeScript chạy trực tiếp trên máy chủ Node.js Render, ứng dụng kỹ thuật *Graph Context Expansion* nạp nhanh dữ liệu phả hệ vào bộ nhớ đệm, giúp phản hồi câu hỏi tức thì cho người dùng 24/7 với chi phí vận hành 0 đồng.

5 Đóng gói & Triển khai

Hệ thống **Portal Họ Lê Văn - Phái 4 - Chi 2** hỗ trợ cả hai phương thức triển khai: đóng gói đa container chạy cục bộ phục vụ phát triển/kiểm thử, và triển khai phân tán hoàn toàn trên nền tảng đám mây (*Cloud Serverless*) với chi phí 0 đồng.

5.1 Đóng gói & Khởi chạy Local bằng Docker

Hệ thống được đóng gói thành các container độc lập và điều phối tập trung thông qua `docker-compose.yml`, bao gồm 4 dịch vụ chính cùng hoạt động trong mạng nội bộ `portal-network`:

- **PostgreSQL (`portal_postgres`):** Sử dụng image `postgres:15-alpine`, tự động khởi tạo cơ sở dữ liệu phả hệ cục bộ tại cổng `5432`.
- **Backend API (`portal_backend`):** Đóng gói môi trường Node.js 22 LTS, biên dịch TypeScript và lắng nghe tại cổng `5000`.
- **Frontend (`portal_frontend`):** Áp dụng kỹ thuật *Multi-stage build* (biên dịch mã nguồn React qua Vite, sau đó chuyển giao tệp tĩnh sang máy chủ **Nginx**) phục vụ người dùng tại cổng `3000`.
- **RAG Service (`portal_rag`):** Đóng gói môi trường Python 3.11-slim, ChromaDB và FastAPI phục vụ tìm kiếm ngữ nghĩa tại cổng `8000`.

Quy trình khởi chạy chỉ với 3 lệnh:

1. Sao chép cấu hình môi trường: `cp .env.example .env`
2. Biên dịch và khởi chạy toàn bộ 4 dịch vụ ở chế độ chạy nền:

```
docker compose up -d --build
```

3. Truy cập trải nghiệm ngay tại địa chỉ: `http://localhost:3000`

5.2 Triển khai Cloud bằng Supabase & Render & Vercel

Nhằm đảm bảo hệ thống vận hành liên tục 24/7 trên Internet với độ trễ thấp và tối ưu chi phí, dự án áp dụng mô hình kiến trúc đám mây phi máy chủ (*Decoupled Cloud Hosting*):

Phân hệ	**Nền tảng**	**Cơ chế / Build Command**	**Địa chỉ truy cập**
Database & Storage	Supabase	PostgreSQL (SSL Pooler) kết hợp Bucket `uploads`	*Quản trị Cloud*
Backend API	Render	`npm install && npm run build` (Node.js 22)	`portal-holevan-phai4-chi2.onrender.com`
Frontend SPA	Vercel	Tự động CI/CD khi đẩy mã nguồn lên GitHub (Vite SPA)	`portal-holevan-phai4-chi2.vercel.app`

Table 1: Danh mục các dịch vụ triển khai trên môi trường Production

Quy trình 3 bước triển khai Production:

1. **Khởi tạo Supabase:** Tạo project mới, lấy chuỗi kết nối `DATABASE_URL` (chế độ Transaction Pooling), tạo Public Bucket `uploads` để lưu ảnh chân dung và tư liệu.
2. **Triển khai Backend trên Render:** Tạo Web Service kết nối kho chứa GitHub (thư mục gốc `backend`), nạp các biến môi trường bí mật (`DATABASE_URL`, `JWT_SECRET`, `GEMINI_API_KEY`, `SUPABASE_URL`) và kích hoạt dịch vụ.
3. **Triển khai Frontend trên Vercel:** Tạo dự án mới liên kết thư mục `frontend`, thiết lập biến môi trường duy nhất `VITE_API_URL` trỏ về địa chỉ Backend Render. Hệ thống Vercel tự động đóng gói và phân phối toàn cầu qua mạng lưới Edge Network.